



API Endpoints — Procurement Hub & Bot Dashboard

Two surfaces, one DB (`getSql()`):

- **Type-safe RPC** (`createServerFn` in `src/lib/plant-server.ts` & `src/lib/bot-server.ts`) — used by React via `useQuery` / `useMutation` .
- **Plain REST** (`src/routes/api/*` for dev + `server/routes/api/*` for prod via Nitro) — for curl, RPA, n8n, external bots. OpenAPI at `GET /api/openapi.json` , interactive explorer at `/system` → **API Endpoints** tab.

Base URL (dev): `http://localhost:8080` — all endpoints are JSON, no auth in this deployment (`VITE_AUTH_ENABLED=false`). When auth is enabled, add `Authorization: Bearer <token>` or cookie and scope by `user_id` .

Health

GET /api/health

Service + DB driver.

Response 200

```
{ "ok": true, "service": "procurement-hub", "version": "1.0.0", "timestamp": "...", "database": "pglite" }
```

cURL

```
curl http://localhost:8080/api/health | jq
```

Catalog

GET /api/catalog

Item & machine master (seeded on first call).

Response

```
{
  "items": [{ "id": 1, "code": "HYD-040", "name": "Hydraulic Oil ISO 68", "uom": "Ltr", "reorderLevel": 20 },
  "machines": [{ "id": 1, "code": "CNC-01", "name": "CNC Lathe 01", "line": "Machine shop" } ]
}
```

ServerFn: `getCatalog()` (GET)

Inventory

GET `/api/inventory`

Rack with 30d consumption, received, `daysCover`, 8-week sparkline.

Row

```
{ "id": 1, "code": "HYD-040", "name": "Hydraulic Oil ISO 68", "uom": "Ltr", "reorderLevel": 20, "qty": 4, "
```

ServerFn: `getInventory()` (GET)

Machines

GET `/api/machines`

Per-machine consumption & breakdown.

Response

```
[{ "id": 1, "code": "CNC-01", "name": "CNC Lathe 01", "line": "Machine shop", "consumed30": 19, "distinct"
```

ServerFn: `getMachineStats()` (GET)

Refill

GET `/api/refill`

Refill dashboard — per-item + weekly series.

Response

```
{ "rows": [{ "itemId": 1, "name": "Hydraulic Oil ISO 68", "code": "HYD-040", "qty": 4, "reorderLevel": 20, "
```

ServerFn: `getRefillDashboard()` (GET)

Slips

GET /api/slips + **GET /api/slips?status=pending**

List slips. Filter by `status=pending|issued|partial|pr_open|received`.

cURL

```
curl "http://localhost:8080/api/slips?status=pending" | jq
curl http://localhost:8080/api/slips | jq '[0]'
```

ServerFn: `listSlips()` (GET)

POST /api/slips

Raise a slip. Requires `hodConfirmed: true`.

Body

```
{"itemId":1,"machineId":1,"qty":5,"department":"Production","station":"Lathe cell","hodTitle":
```

Validation: `itemId`, `machineId`, `qty` (int >=1), `hodConfirmed`.

Response 200 — created slip.

cURL

```
curl -X POST http://localhost:8080/api/slips -H 'content-type: application/json' -d '{"itemId":
```

ServerFn: `raiseSlip({data:{...}})` (POST)

POST /api/slips/decide

Store decision for a pending slip.

Body

```
{"id":1,"mode":"stock"} // stock | split | pr
```

- `stock` — issue full qty (fails if `onHand < qty`, use `split`)
- `split` — issue `min(qty, onHand)`, open PR for remainder
- `pr` — no issue, open PR

Side effects: atomically decrements `items.qty`, inserts `movements` (`kind=issue`, `qty=-want`), updates `slips` to `issued` / `partial` / `pr_open`.

cURL

```
curl -X POST http://localhost:8080/api/slips/decide -H 'content-type: application/json' -d '{
```

ServerFn: `decideSlip({data:{id, mode}})` (POST)

POST /api/slips/receive

Mark an open PR as received. Increments stock.

Body

```
{ "id": 2 }
```

cURL

```
curl -X POST http://localhost:8080/api/slips/receive -H 'content-type: application/json' -d '{
```

ServerFn: `receiveSlip({data:{id}})` (POST)

POST /api/slips → **GET /api/inventory loop**

After `decide / receive`, `inventory.qty` and `consumed30` reflect immediately (no cache staleness — `staleTime: 4s`).

Bots

GET /api/bots + **GET /api/bots?status=active&type=inventory**

List bots. Filters: `status=active|paused|error|draft`, `type=inventory|slip|reorder|report|generic`.

cURL

```
curl http://localhost:8080/api/bots | jq '.[0]'  
curl "http://localhost:8080/api/bots?status=active" | jq
```

ServerFn: `listBots()` (GET), `getBot({data:{id}})` (GET)

POST /api/bots

Create bot.

Body

```
{ "code": "BOT-TEST", "name": "Test Bot", "description": "demo", "type": "generic", "status": "draft", "c
```

Validation: `code` must match `^[A-Z0-9-]+$`.

cURL

```
curl -X POST http://localhost:8080/api/bots -H 'content-type: application/json' -d '{"code":"B
```

ServerFn: `createBot({data:{...}})` (POST), `updateBot({data:{id,...}})`, `deleteBot({data:{id}})`

POST /api/bots/trigger

Manual run. Fails if `status=paused`.

Body

```
{"id":1,"input":{}}
```

Response — created `bot_runs` row with `trigger=manual`, random `duration_ms`.

cURL

```
curl -X POST http://localhost:8080/api/bots/trigger -H 'content-type: application/json' -d '{"
```

ServerFn: `triggerBot({data:{id,input}})` (POST)

Runs

GET /api/runs + **GET /api/runs?botId=1&limit=20**

Bot execution history, newest first.

cURL

```
curl "http://localhost:8080/api/runs?limit=5" | jq
curl "http://localhost:8080/api/runs?botId=1&limit=10" | jq
```

Row

```
{"id":1,"botId":1,"botCode":"BOT-INV-SYNC","botName":"Inventory Sync Bot","status":"success","
```

ServerFn: `listBotRuns({data:{botId,limit}})` (GET), `getBotStats()` (GET) for aggregates.

Alerts

GET /api/alerts + filters

List alerts.

Query: `severity=info|warn|critical`, `acknowledged=true|false`, `limit=1..100`.

cURL

```
curl http://localhost:8080/api/alerts | jq
curl "http://localhost:8080/api/alerts?severity=critical" | jq
curl "http://localhost:8080/api/alerts?acknowledged=false&limit=10" | jq
```

ServerFn: `listAlerts({data:{...}})` (GET)

POST /api/alerts

Create alert.

Body

```
{"severity":"warn","title":"Low stock: Hydraulic Oil","message":"Qty 4 vs reorder 20","botId":
```

ServerFn: `createAlert({data:{...}})` (POST)

POST /api/alerts/acknowledge

Acknowledge.

Body

```
{"id":1}
```

cURL

```
curl -X POST http://localhost:8080/api/alerts/acknowledge -H 'content-type: application/json'
```

ServerFn: `acknowledgeAlert({data:{id}})` (POST)

OpenAPI

GET /api/openapi.json

OpenAPI 3.0 spec for all endpoints. Also served as `server/routes/api/openapi.json.ts` (prod) and `src/routes/api/openapi[.]json.ts` (dev).

cURL

```
curl http://localhost:8080/api/openapi.json | jq '.info'
```

Interactive docs: `/system` → **API Endpoints** tab has Try-it + curl copy.

Server Functions (RPC) vs REST

USE	CALL	TRANSPORT
React loaders/components	<code>getCatalog()</code> / <code>listSlips()</code> etc.	<code>createServerFn</code> → <code>_serverFn</code> RPC, type-safe, no fetch
External RPA/curl/n8n	<code>GET /api/catalog</code> etc.	<code>src/routes/api/*</code> (dev) + <code>server/routes/api/*</code> (prod) → <code>Response.json</code>

Both share `getSql()` and `ensureSeed()` so seed/migrations are identical.

Error Format

All REST errors are JSON with `error` :

```
{"error":"HOD authorisation required"}
```

Status codes: `400` for validation, `404` for not found, `409` for conflict (e.g., stock moved), `405` for wrong method.

Auth (when enabled)

- Set `VITE_AUTH_ENABLED != "false"` in `.grok/app-env.json`
- Copy `migrations/auth/0001_auth.sql` → `migrations/0001_auth.sql`
- Server functions then use `requireUserId()` to scope queries by `user_id` ; REST handlers check `Authorization: Bearer` via `getRequest()` .

Examples — end-to-end

```
# 1. Raise a slip for Carbide Insert (id 2) on CNC Lathe (id 1)
curl -X POST http://localhost:8080/api/slips -H 'content-type: application/json' -d '{"itemId"
# → SLIP-2609-0007

# 2. Store decides — stock is 0, so PR
curl -X POST http://localhost:8080/api/slips/decide -H 'content-type: application/json' -d '{"
# → pr_open

# 3. Mark PR received (increments stock)
curl -X POST http://localhost:8080/api/slips/receive -H 'content-type: application/json' -d '{"

# 4. Verify inventory updated
curl http://localhost:8080/api/inventory | jq '.[[] | select(.code=="CUT-118") | .qty'
```

```
# 5. Trigger a bot
curl -X POST http://localhost:8080/api/bots/trigger -H 'content-type: application/json' -d '{}'
```